Name : Vatsal Postiwala PRN No: 26070122509 Batch : B3(2025-2029)

Subject: Programming with Java

ASSIGNMENT No: 22

TITLE: Demonstration and exploration of JDBC classes and interfaces (PreparedStatement, and ResultSet).

Problem Statement

Demonstration and exploration of JDBC classes and interfaces (PreparedStatement, and ResultSet).

Learning Objectives

To implement PreparedStatement and ResultSet for secure query execution and data retrieval.

Theory

PreparedStatement is an advanced JDBC interface used to execute precompiled SQL statements with parameters. Unlike the Statement interface, PreparedStatement allows SQL queries to be compiled only once and executed multiple times with different values, thereby improving performance. It also protects applications against SQL injection attacks by separating SQL code from user input. The ResultSet interface stores the records returned by SQL SELECT queries. It provides various methods such as next(), getInt(), getString(), and getDouble() for accessing retrieved data. PreparedStatement and ResultSet together provide secure, efficient, and flexible database interaction and are widely used in enterprise Java applications.

Code:

```
package E22;

import java.lang.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class E22 {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/sit_java";
        String username = "root";
```

```
        String password = "12345";

        try (Connection connection = DriverManager.getConnection(url, username, password)) {

            String insertQuery = "INSERT INTO students (id, name) VALUES (?, ?)";
            try (PreparedStatement insertStmt = connection.prepareStatement(insertQuery)) {
                insertStmt.setInt(1, 101);
                insertStmt.setString(2, "Aarav");
                insertStmt.executeUpdate();
            }

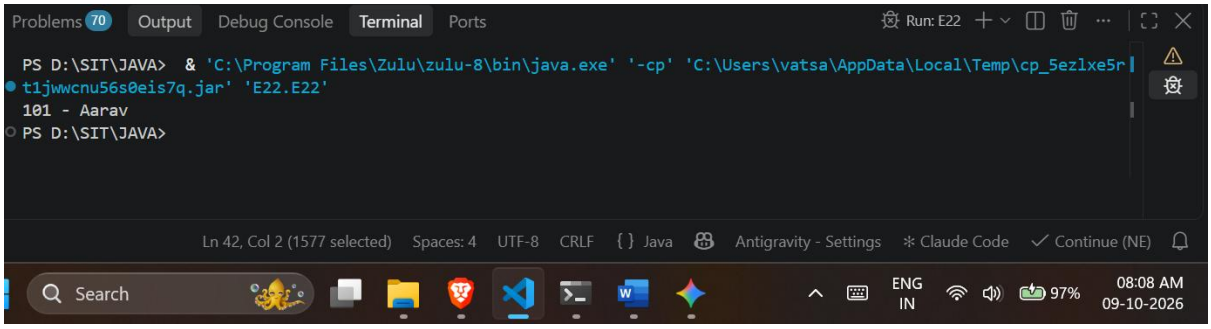
            String selectQuery = "SELECT id, name FROM students WHERE id = ?";
            try (PreparedStatement selectStmt = connection.prepareStatement(selectQuery)) {
                selectStmt.setInt(1, 101);

                try (ResultSet resultSet = selectStmt.executeQuery()) {
                    while (resultSet.next()) {
                        int studentId = resultSet.getInt("id");
                        String studentName = resultSet.getString("name");

                        System.out.println(studentId + " - " + studentName);
                    }
                }
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Output:



Exercise

1. Create a login application using PreparedStatement to check username and password.

Code:

```
package E22;

import java.lang.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Scanner;

public class LoginApp {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/sit_java";
        String dbUser = "root";
        String dbPassword = "12345";

        try (Scanner scanner = new Scanner(System.in);
             Connection connection = DriverManager.getConnection(url, dbUser, dbPassword)) {

            System.out.print("Enter username: ");
            String username = scanner.nextLine();
            System.out.print("Enter password: ");
            String password = scanner.nextLine();

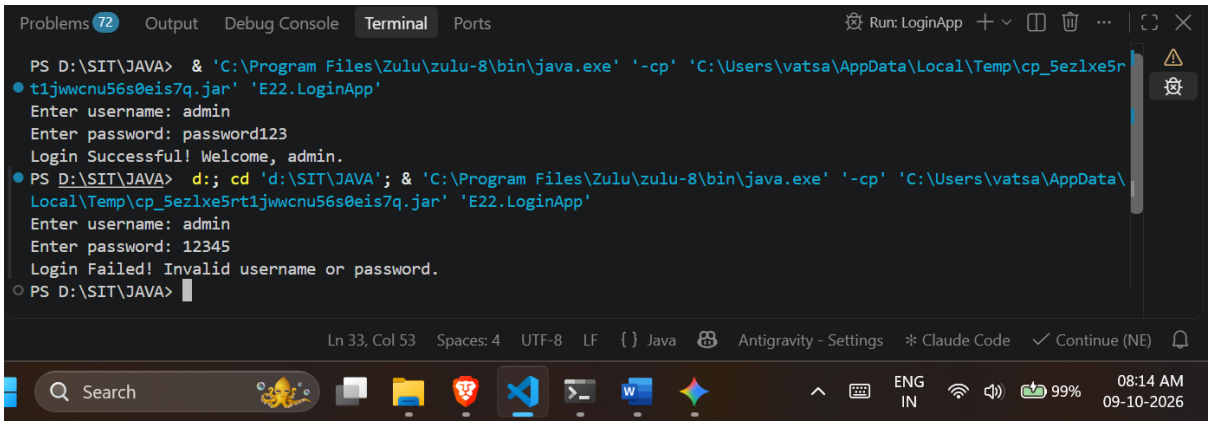
            String query = "SELECT * FROM users WHERE username = ? AND password = ?";

            try (PreparedStatement pstmt = connection.prepareStatement(query)) {
                pstmt.setString(1, username);
                pstmt.setString(2, password);

                try (ResultSet rs = pstmt.executeQuery()) {
                    if (rs.next()) {
                        System.out.println("Login Successful! Welcome, " + username + ".");
                    } else {
                        System.out.println("Login Failed! Invalid username or password.");
                    }
                }
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
```

}

Outout:



2. Create a Java application using **PreparedStatement** to authenticate **hospital staff login** (doctor/nurse/login ID and password). Display appropriate access messages based on authentication result.

Code:

```
package E22;

import java.lang.*;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Scanner;

public class HospitalLoginApp {
   public static void main(String[] args) {
      String url = "jdbc:mysql://localhost:3306/sit_java";
      String dbUser = "root";
      String dbPassword = "12345";

      try (Scanner scanner = new Scanner(System.in);
            Connection connection = DriverManager.getConnection(url, dbUser, dbPassword)) {

         System.out.print("Enter Staff Login ID: ");
         String loginId = scanner.nextLine();
```

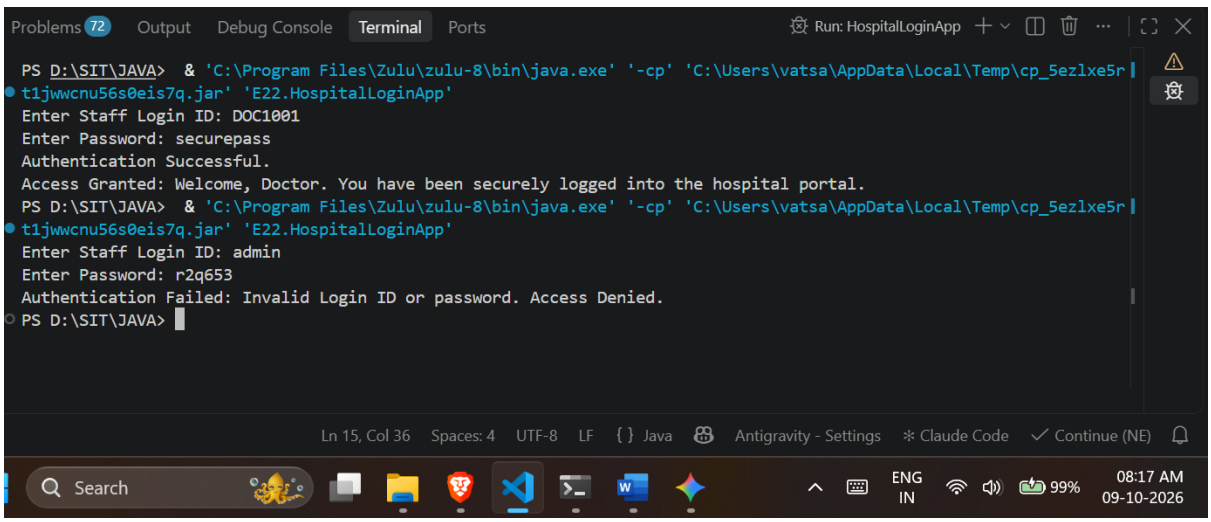
```
        System.out.print("Enter Password: ");
        String password = scanner.nextLine();

        String query = "SELECT role FROM hospital_staff WHERE login_id = ? AND password = ?";

        try (PreparedStatement pstmt = connection.prepareStatement(query)) {
            pstmt.setString(1, loginId);
            pstmt.setString(2, password);

            try (ResultSet rs = pstmt.executeQuery()) {
                if (rs.next()) {
                    String role = rs.getString("role");
                    System.out.println("Authentication Successful.");
                      System.out.println("Access Granted: Welcome, " + role + ". You have been securely logged into the hospital portal.");
                } else {
                      System.out.println("Authentication Failed: Invalid Login ID or password. Access Denied.");
                }
            }
        } catch (SQLException e) {
            System.out.println("Database error: " + e.getMessage());
        }
    }
}
```

Outout:



Conclusion:

I learned how to establish a secure connection between a Java application and a MySQL database using JDBC. Furthermore, I learned to utilize the PreparedStatement interface for executing parameterized queries, which efficiently handles user authentication and prevents SQL injection. Lastly, I learned how to retrieve and evaluate database records using the ResultSet interface to build robust login systems.